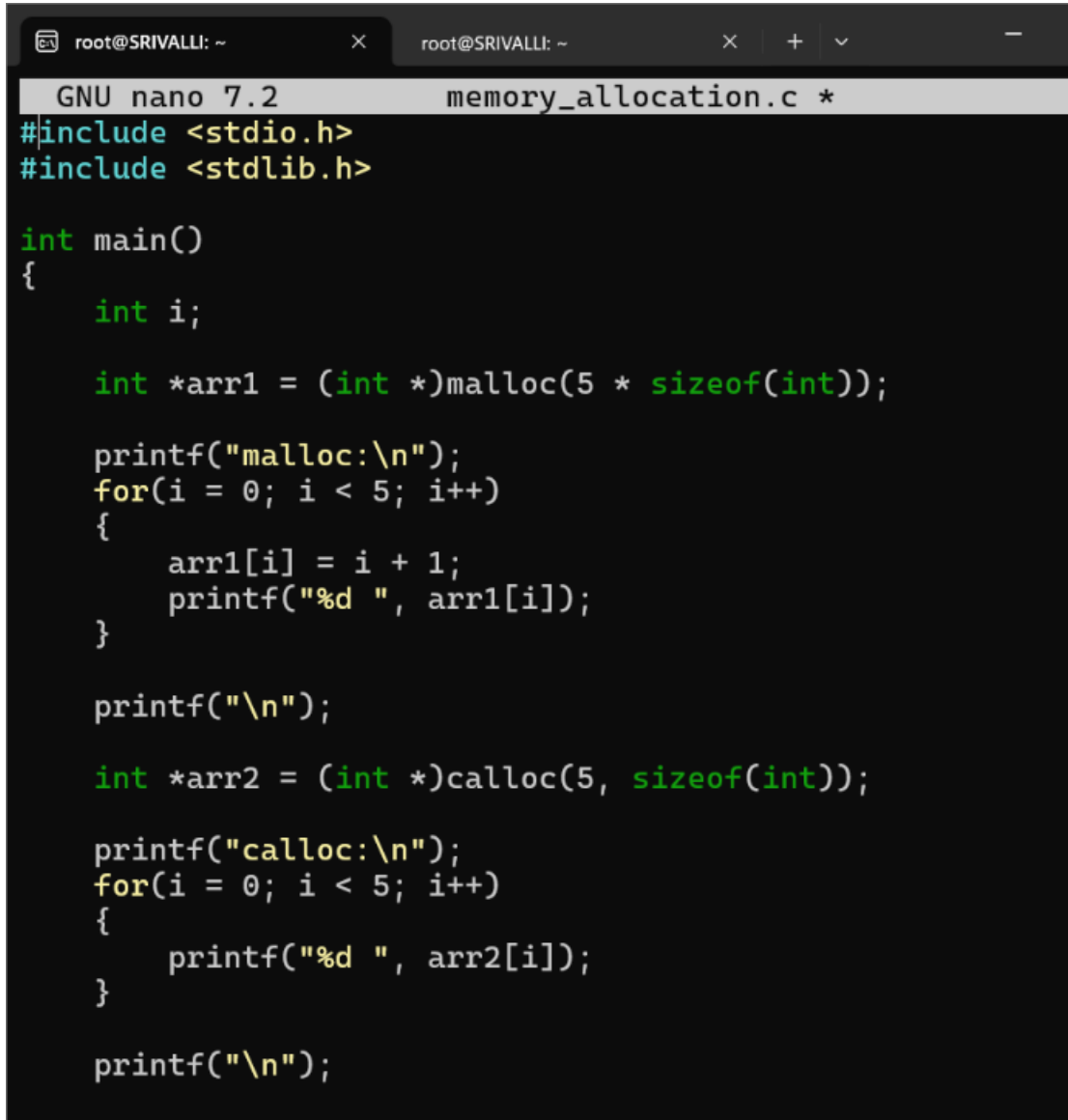


PRACTICAL-08

1. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

CODE:

A screenshot of a terminal window with a dark background. At the top, there are two tabs labeled 'root@SRIVALLI: ~'. Below the tabs, the title bar of the nano editor shows 'GNU nano 7.2' and 'memory_allocation.c *'. The code is written in a light green and yellow monospace font. It includes <stdio.h> and <stdlib.h>. The main function starts with an integer i, then allocates an array arr1 using malloc(5 * sizeof(int)). It prints 'malloc:\n', then loops from i=0 to i=4, setting arr1[i] = i + 1 and printing '%d ' for each element. After a blank line, it allocates an array arr2 using calloc(5, sizeof(int)). It prints 'calloc:\n', then loops from i=0 to i=4, printing '%d ' for each element. Finally, it prints a blank line.

```
root@SRIVALLI: ~
root@SRIVALLI: ~
GNU nano 7.2      memory_allocation.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    int *arr1 = (int *)malloc(5 * sizeof(int));

    printf("malloc:\n");
    for(i = 0; i < 5; i++)
    {
        arr1[i] = i + 1;
        printf("%d ", arr1[i]);
    }

    printf("\n");

    int *arr2 = (int *)calloc(5, sizeof(int));

    printf("calloc:\n");
    for(i = 0; i < 5; i++)
    {
        printf("%d ", arr2[i]);
    }

    printf("\n");
}
```

```
        printf("%d ", arr1[i]);
    }

    printf("\n");

    int *arr2 = (int *)calloc(5, sizeof(int));

    printf("calloc:\n");
    for(i = 0; i < 5; i++)
    {
        printf("%d ", arr2[i]);
    }

    printf("\n");

    arr1 = (int *)realloc(arr1, 10 * sizeof(int));

    printf("realloc successful\n");

    free(arr1);
    free(arr2);

    printf("Memory freed successfully\n");

    return 0;
```